ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



HỌC SÂU VÀ ỨNG DỤNG
TRONG THỊ GIÁC MÁY TÍNH
Mã môn học: CO5085

BÁO CÁO BÀI TẬP E1–E3

Phân loại ảnh với CNN, attention
và mạng hồi tiếp

Học kỳ 261 · Năm học 2026–2027

Giảng viên:	Lê Thành Sách
	Nguyễn Quốc Minh
Sinh viên:	Huỳnh Thành Đạt
Mã số sinh viên:	2570161
Hình thức:	Cá nhân

TP. Hồ Chí Minh, tháng 10 năm 2026

Mục lục

1 Dữ liệu và quy trình chung **1**
1.1 Phạm vi bài làm . . . 1
1.2 Tập dữ liệu và cách chia . . . 1
1.3 Huấn luyện và chỉ số đánh giá . . . 2

2 E1: Softmax, MLP và CNN **3**
2.1 Ba bộ phân loại . . . 3
2.2 Kết quả và phân tích lỗi . . . 3

3 E2: MSA và cách lấy token **7**
3.1 Hiện thực attention . . . 7
3.2 Ba cách tokenization . . . 7
3.3 So sánh attention và tokenization . . . 8

4 E3: LSTM/GRU trên chuỗi ảnh **11**
4.1 Biểu diễn ảnh và kiến trúc . . . 11
4.2 Các phương trình được sử dụng . . . 11
4.3 Kết quả và quan hệ không gian . . . 12

5 Mở rộng E3: Điều kiện truyền gradient **14**
5.1 Vấn đề và giả thuyết . . . 14
5.2 Khởi tạo và regularization trực giao . . . 14
5.3 Bias cổng nhớ theo độ dài chuỗi . . . 15
5.4 Mean pooling và clipping . . . 15
5.5 Ablation và cách kiểm tra . . . 16

6 Tái lập và giới hạn **19**
6.1 Mã nguồn và tệp kết quả . . . 19
6.2 Cấu hình và các lượt chạy . . . 19
6.3 Kết luận . . . 20

Tài liệu tham khảo **21**

Chương 1

Dữ liệu và quy trình chung

1.1 Phạm vi bài làm

Báo cáo so sánh các mô hình phân loại ảnh trên cùng tập Fashion-MNIST. E1 gồm softmax, MLP và CNN. E2 trình bày hiện thực multi-head self-attention (MSA), đối chiếu với thư viện và khảo sát ba cách tạo token. E3 biểu diễn ảnh thành chuỗi để phân loại bằng LSTM/GRU, với MLP và CNN của E1 làm đối chứng. Chỉ số bước của mạng hồi tiếp tương ứng với thứ tự quét các vùng trên ảnh.

Phần mở rộng khảo sát bốn thay đổi trên LSTM: khởi tạo ma trận hồi quy trực giao, regularization trực giao mềm, khởi tạo bias cổng theo độ dài chuỗi và mean pooling. Mỗi thành phần được đánh giá riêng, sau đó kết hợp và thực hiện ablation. Tất cả mô hình trong E1–E3 đều được huấn luyện từ đầu.

Thực nghiệm gồm 20 cấu hình, mỗi cấu hình chạy với ba seed 42, 43 và 44, tương ứng 60 lượt huấn luyện. Các mô hình dùng chung cách chia dữ liệu và quy tắc chọn checkpoint để so sánh kết quả.

1.2 Tập dữ liệu và cách chia

Fashion-MNIST gồm ảnh thời trang mức xám 28×28, 10 lớp, với 60.000 ảnh train và 10.000 ảnh test chính thức [2]. Kích thước ảnh nhỏ phù hợp để khảo sát cả CNN, attention và mạng hồi tiếp trên cùng đầu vào.

Từ 60.000 ảnh train, tách phân tầng 10% làm validation với split seed 2026. Như vậy có 54.000 ảnh train, 6.000 validation và 10.000 test. Mỗi lớp lần lượt có 5.400, 600 và 1.000 ảnh theo phân bố chuẩn của bộ dữ liệu. Split seed không đổi giữa E1–E3 và không trùng vai trò với seed khởi tạo mô hình.

Ảnh được chuyển từ uint8 sang $[0, 1]$, sau đó chuẩn hóa $x' = (x - \mu)/\sigma$. Hai thống kê μ, σ được tính trên 54.000 ảnh train và dùng lại cho validation/test. Các thí nghiệm không dùng augmentation để giữ đầu vào thống nhất khi so sánh cách biểu diễn ảnh.

`src/common/data.py` lưu chỉ số từng split, checksum dữ liệu và fingerprint của cả split lẫn normalization. Loader ưu tiên HTTPS từ repo chính thức của dataset và giữ kiểm tra checksum của torchvision. Dữ liệu test có thể được tải cùng kho ảnh, nhưng không được đọc vào vòng chọn checkpoint hay tính normalization.

1.3 Huấn luyện và chỉ số đánh giá

Mỗi mô hình xuất logits $z \in \mathbb{R}^{10}$. Với nhãn đúng y, cross-entropy là

$$\ell(z, y) = -z_y + \log \sum_{k=1}^{10} \exp(z_k), \qquad \frac{\partial \ell}{\partial z_k} = p_k - \mathbf{1}[k = y]. \tag{1.1}$$

Vì `cross_entropy` đã thực hiện log-softmax ổn định số, code không chèn softmax trước loss. Softmax chỉ được dùng khi cần xác suất để trình bày dự đoán.

Thiết lập	Giá trị định trước
Optimizer	AdamW, weight decay 10^{-4} trên các tham số trainable
Ngân sách	30 epoch, batch 256, không early stopping
Learning rate	E1/E3: 10^{-3}; E2: 3×10^{-4}
Scheduler	Cosine decay về 1% learning rate ban đầu
Gradient clipping	Global norm tối đa 1, sau backward và trước step
Độ chính xác số	FP32; tắt AMP, TF32 và compile
Seed mô hình	42, 43, 44; cùng split seed 2026
Chọn checkpoint	Validation accuracy lớn nhất; hòa thì CE nhỏ nhất

Bảng 1.1: Thiết lập huấn luyện dùng chung cho E1–E3.

Vòng lặp huấn luyện gồm zero gradient, forward, tính loss, backward, clipping và optimizer step. Checkpoint được chọn trên validation; test được đánh giá sau khi hoàn tất các cấu hình đã định. Chương trình kiểm tra tính hữu hạn của loss và gradient tại mỗi bước cập nhật.

Accuracy là chỉ số chính; macro-F1 bổ sung đánh giá theo lớp. Confusion matrix và các dự đoán sai có confidence cao được dùng để phân tích lỗi. Chi phí mô hình được mô tả qua số tham số và thời gian train/epoch. Thời gian được đồng bộ CUDA, gồm lấy batch và cập nhật trọng số; median bỏ epoch đầu để giảm ảnh hưởng khởi tạo. Tốc độ inference được đo riêng, bỏ hai batch đầu và không tính thời gian truyền dữ liệu sang GPU.

Bảng kết quả trình bày độ chính xác xác thực trung bình và độ chính xác kiểm tra dưới dạng trung bình $\pm$ độ lệch chuẩn mẫu qua ba seed. Các đường cong, confusion matrix và ảnh minh họa dùng seed 42. Ngân sách 30 epoch được giữ cố định cho mọi cấu hình; do đó kết quả phản ánh hiệu quả trong ngân sách này.

Chương 2

E1: Softmax, MLP và CNN

2.1 Ba bộ phân loại

Softmax nhận vector $x \in \mathbb{R}^{784}$ và tính logits $z = Wx + b$; ranh giới giữa hai lớp là siêu phẳng trong không gian pixel. MLP bổ sung hai tầng phi tuyến trên cùng vector flatten. CNN dùng kernel chia sẻ theo vị trí để học đặc trưng cục bộ. Ba kiến trúc được giữ nhỏ và mô tả đầy đủ trong bảng dưới đây.

Mô hình	Các lớp theo thứ tự
Softmax	Flatten $784 \to 10$
MLP	Flatten; $784 \to 256$; ReLU; dropout 0,2; $256 \to 128$; ReLU; dropout 0,2; $128 \to 10$
CNN	Conv $1 \to 32$, kernel 3, padding 1; BN; ReLU; pool 2; Conv $32 \to 64$ tương tự; pool 2; flatten; $3136 \to 128$; ReLU; dropout 0,2; $128 \to 10$

Bảng 2.1: Kiến trúc E1. Conv không dùng bias vì sau đó có batch normalization.

Hai phép pooling đưa ảnh $28 \to 14 \to 7$, vì vậy head CNN nhận $64 \times 7 \times 7 = 3136$ giá trị. Với một lớp Linear $d_{in} \to d_{out}$ có bias, số tham số là $d_{in}d_{out} + d_{out}$. Ví dụ softmax có $784 \times 10 + 10 = 7.850$ tham số. Các số còn lại được đếm trực tiếp bằng `numel()`, gồm tham số affine của BN nhưng không gồm running statistics.

2.2 Kết quả và phân tích lỗi

Mô hình	Tham số	Val (%)	Test (%)	Giây/epoch
Softmax	7,850	85.98	84.51 ± 0.15	1.51
MLP	235,146	91.02	89.88 ± 0.14	1.67
CNN	421,738	93.03	92.69 ± 0.15	1.97

Bảng 2.2: Kết quả E1, tổng hợp theo các seed được công bố.

Val/Test: accuracy (%). Tham số đếm từ kiến trúc; $\pm$: SD qua ba seed. Các lượt chạy được liệt kê ở Chương 6; cách đo thời gian ở Chương 1.

Train/validation CE được đối chiếu với accuracy để phân tích quá trình học. Confusion matrix và ảnh dự đoán sai bên dưới lấy từ CNN ở seed 42. Thí nghiệm dùng ba hạt giống 42, 43 và 44 trên NVIDIA GeForce RTX 5090 với PyTorch 2.10.0+cu130 và CUDA runtime 13.0. Chênh

lệch giữa hai cấu hình được tính theo cặp cùng hạt giống, rồi lấy trung bình và độ lệch chuẩn mẫu.

Softmax đạt độ chính xác kiểm tra $(84.51 \pm 0.15)\%$, và macro-F1 0.8436. Cross-entropy trung bình ở epoch cuối trên tập huấn luyện và xác thực lần lượt là 0.3663 và 0.4095.

MLP đạt độ chính xác kiểm tra $(89.88 \pm 0.14)\%$, và macro-F1 0.8985. Cross-entropy trung bình ở epoch cuối trên tập huấn luyện và xác thực lần lượt là 0.1371 và 0.2832.

CNN đạt độ chính xác kiểm tra $(92.69 \pm 0.15)\%$, và macro-F1 0.9267. Cross-entropy trung bình ở epoch cuối trên tập huấn luyện và xác thực lần lượt là 0.0174 và 0.2969.

MLP trừ Softmax: +5.37 $\pm$ 0.29 điểm phần trăm.

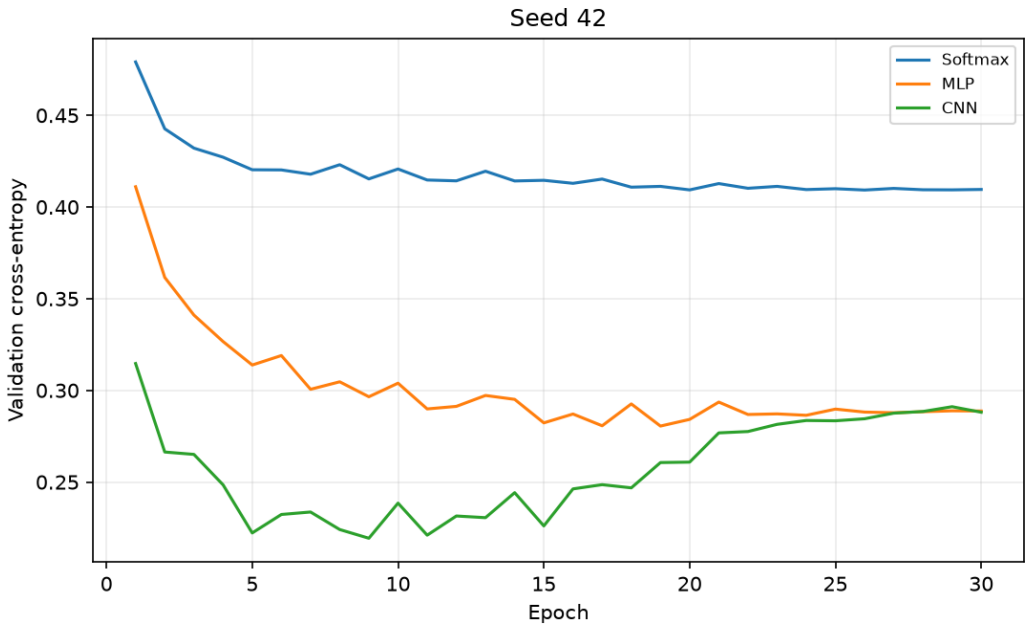
CNN trừ MLP: +2.81 $\pm$ 0.01 điểm phần trăm.

MLP bổ sung các lớp phi tuyến. CNN giữ cấu trúc hai chiều, chia sẻ trọng số tích chập và gộp cục bộ. Xếp hạng đã đo phù hợp với lợi ích của các biểu diễn này trên ảnh nhỏ. Khác biệt về số tham số và regularization cũng góp phần vào kết quả của từng kiến trúc.

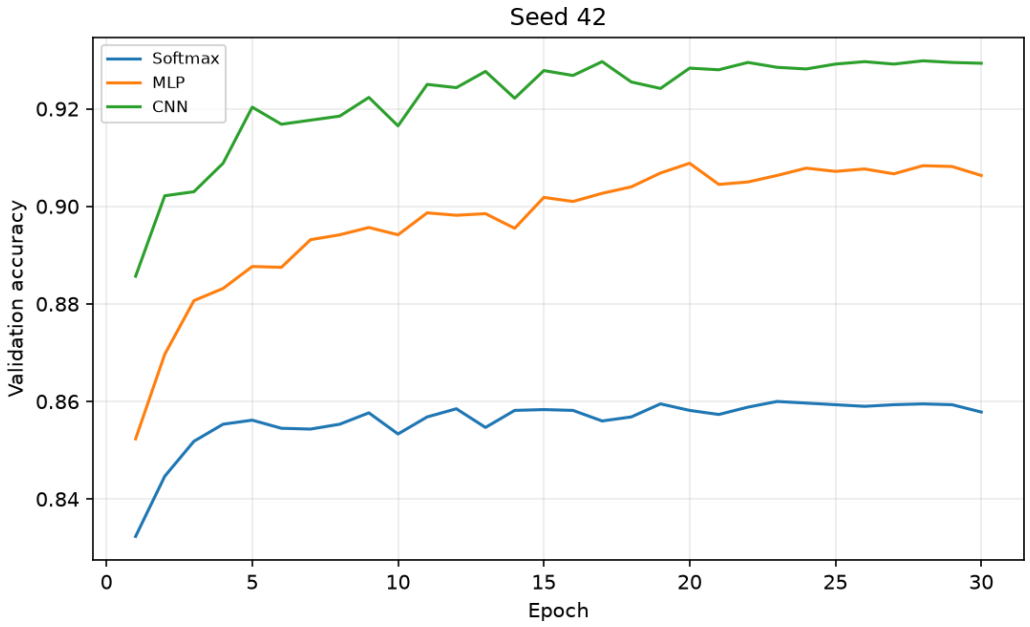
CNN đạt độ chính xác kiểm tra cao nhất, đồng thời có khoảng cách cross-entropy lớn giữa tập huấn luyện và xác thực ở epoch 30. Các dự đoán sai có độ tin cậy cao cho thấy mô hình vẫn có thể gán xác suất lớn cho lớp nhầm. Kết quả kiểm tra được tính tại checkpoint đã chọn bằng tập xác thực.

Ở CNN seed 42, lỗi nhiều nhất theo một chiều là Shirt thành T-shirt/top: 90 ảnh.

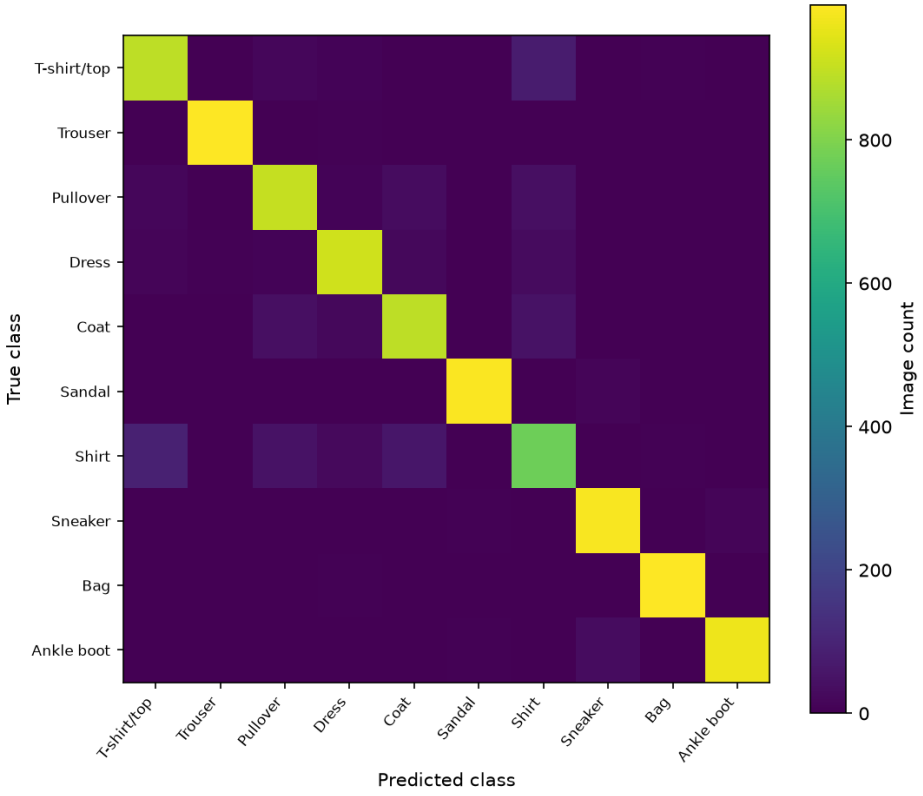
Kiểm tra bổ sung dùng điểm tin cậy của lớp dự đoán trong các tệp đã lưu. ECE được tính bằng cách chia điểm tin cậy thành 15 khoảng đều, lấy chênh lệch tuyệt đối giữa độ chính xác và điểm tin cậy trung bình trong mỗi khoảng, rồi gộp theo số ảnh. ECE trung bình của softmax, MLP và CNN lần lượt là 0,96, 3,03 và 4,47 điểm phần trăm. CNN phân loại đúng nhiều nhất nhưng điểm tin cậy trung bình 97,13% cao hơn độ chính xác 92,69%. Kết quả trên cùng tập kiểm tra này tách hai thuộc tính: chọn đúng lớp và gán mức tin cậy phù hợp.



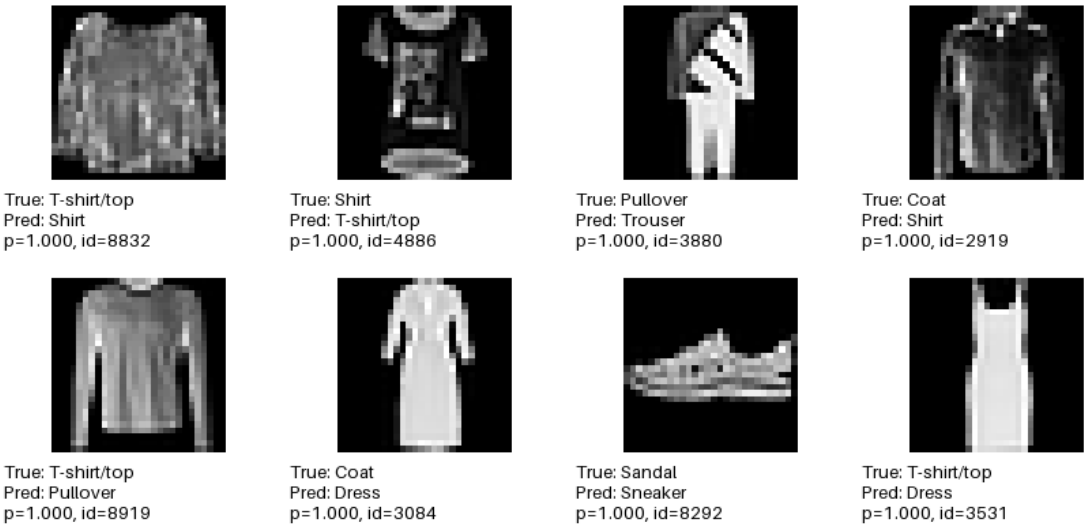
Hình 2.1: Validation CE của ba mô hình E1 ở seed 42.



Hình 2.2: Validation accuracy của E1; seed dùng cho hình ghi trên biểu đồ.



Hình 2.3: Confusion matrix test của CNN ở seed đại diện.



Hình 2.4: Một số dự đoán sai có confidence cao của CNN.

Chương 3

E2: MSA và cách lấy token

3.1 Hiện thực attention

Với tensor token $X \in \mathbb{R}^{B\times N\times d}$, mỗi head học các phép chiếu query, key và value. Bỏ chỉ số batch để gọn, attention của một head là [3]

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \qquad A = \text{softmax}_{\text{hàng}}\left(\frac{QK^\top}{\sqrt{d_h}}\right), \quad O = AV. \tag{3.1}$$

Các bias được lược bỏ trong công thức cho gọn, nhưng có trong hiện thực. Ở đây $d = 64$, có bốn head nên $d_h = 16$. Ba phép Linear được gộp thành QKV, reshape thành các head, nhân ma trận, softmax theo chiều key, áp dụng dropout, ghép head và chiếu output. Bản manual không gọi `nn.MultiheadAttention`, `TransformerEncoder` hoặc fused attention để thay phần tính này.

Bản đối chiếu dùng `nn.MultiheadAttention(batch_first=True)`. Chương trình tạo toàn bộ model manual rồi copy đúng QKV và output projection sang module PyTorch. Các phần tokenizer, positional embedding, FFN, LayerNorm và head cũng giữ nguyên tensor khởi tạo của cùng seed. Chỉ reset seed trước hai constructor khác nhau là chưa đủ bảo đảm điều này.

Tính tương đương số học được kiểm tra bằng tensor nhỏ và dropout bằng 0, qua output, gradient đầu vào và gradient của toàn bộ tham số. Khi huấn luyện, dropout và kernel có thể làm hai phiên bản đi theo quỹ đạo khác nhau dù cùng khởi tạo.

3.2 Ba cách tokenization

Cách chia	N	Đầu vào/token	Phép tạo embedding
Hàng	28	28	Linear $28 \to 64$
Patch 4×4	49	16	Unfold không chồng lấp, Linear $16 \to 64$
CNN-stem	49	64	Hai Conv stride 2: $1 \to 32 \to 64$, BN/ReLU; flatten spatial

Bảng 3.1: Tokenization của E2 trên ảnh $1 \times 28 \times 28$.

Token hàng gom các pixel cùng một hàng. Token patch quét lưới từ trái sang phải, trên xuống dưới. CNN-stem rút feature map $64 \times 7 \times 7$; mỗi vị trí là một token đã qua xử lý cục bộ. Stem được huấn luyện từ đầu và bổ sung cả phép tính lẫn tham số so với hai cách tạo token trực tiếp.

Mỗi cấu hình thêm learned positional embedding, qua hai encoder block dạng pre-LN và kết

thúc bằng LayerNorm, mean pooling và Linear $64 \to 10$. Không có CLS token. Mỗi block tính

$$U = X + \text{Dropout}(\text{MSA}(\text{LN}(X))), \tag{3.2}$$

$$Y = U + \text{FFN}(\text{LN}(U)), \tag{3.3}$$

trong đó FFN là $64 \to 128 \to 64$ với GELU và dropout 0,1. Attention không dùng causal mask vì toàn bộ ảnh đã có tại thời điểm phân loại.

3.3 So sánh attention và tokenization

Mô hình	Tham số	Val (%)	Test (%)	Giây/epoch
patches / manual	71,946	87.03	85.93 ± 0.60	4.52
patches / torch	71,946	87.09	86.02 ± 0.37	4.46
rows / manual	71,370	88.61	87.92 ± 0.03	4.93
rows / torch	71,370	88.61	87.86 ± 0.02	4.60
cnn / manual	89,770	89.26	88.66 ± 0.40	5.10
cnn / torch	89,770	89.40	88.63 ± 0.35	4.76

Bảng 3.2: Sáu cấu hình E2. Manual và PyTorch của cùng tokenizer có cùng số tham số.

Val/Test: accuracy (%). Tham số đếm từ kiến trúc; ±: SD qua ba seed. Các lượt chạy được liệt kê ở Chương 6; cách đo thời gian ở Chương 1.

Manual và thư viện được so sánh ở từng tokenizer; ba tokenizer được so sánh khi giữ attention backend cố định. Chi phí phần score của attention tỉ lệ $N^2 d$; số cặp token của patch gấp $49^2/28^2 \approx 3,06$ lần token hàng. Thời gian toàn mô hình còn gồm projection, FFN, stem và chi phí thực thi kernel.

Các kết quả dưới đây được đo trên cùng split, với cùng thiết lập giữa hai backend của mỗi tokenizer. Với token theo hàng, chênh lệch độ chính xác kiểm tra giữa bản PyTorch và bản tự viết là -0.06 ± 0.04 điểm phần trăm. Trung bình các trung vị thời gian mỗi epoch của hai bản lần lượt là 4.60 và 4.93 giây.

Với token theo ô ảnh, chênh lệch độ chính xác kiểm tra giữa bản PyTorch và bản tự viết là +0.10 ± 0.23 điểm phần trăm. Trung bình các trung vị thời gian mỗi epoch của hai bản lần lượt là 4.46 và 4.52 giây.

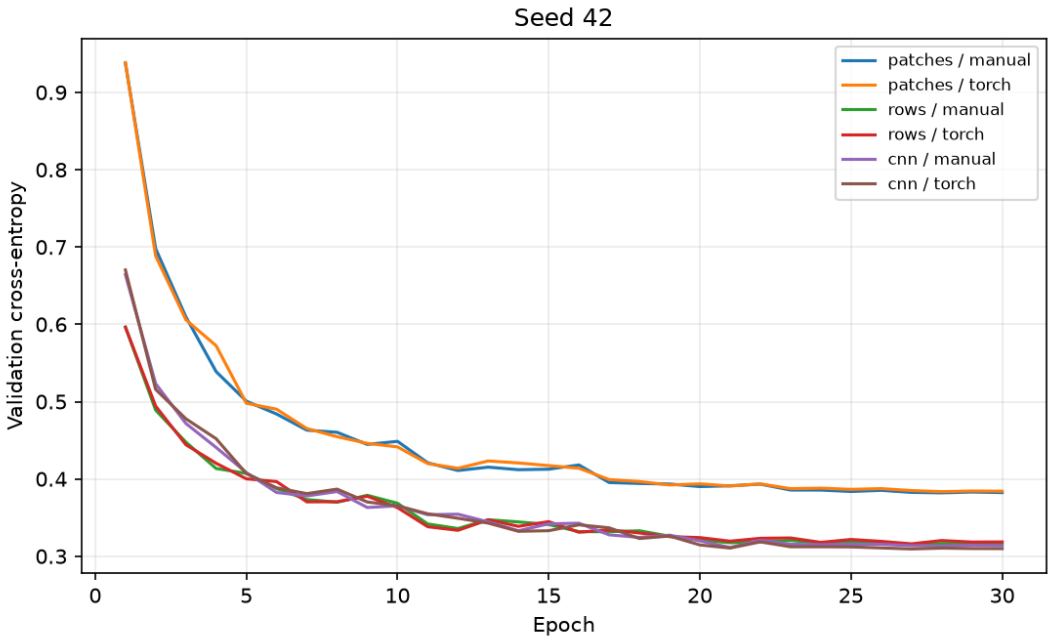
Với đặc trưng CNN-stem, chênh lệch độ chính xác kiểm tra giữa bản PyTorch và bản tự viết là -0.02 ± 0.10 điểm phần trăm. Trung bình các trung vị thời gian mỗi epoch của hai bản lần lượt là 4.76 và 5.10 giây.

Trong sáu cấu hình, bản tự viết với đặc trưng CNN-stem có độ chính xác kiểm tra trung bình cao nhất, đạt 88.66%.

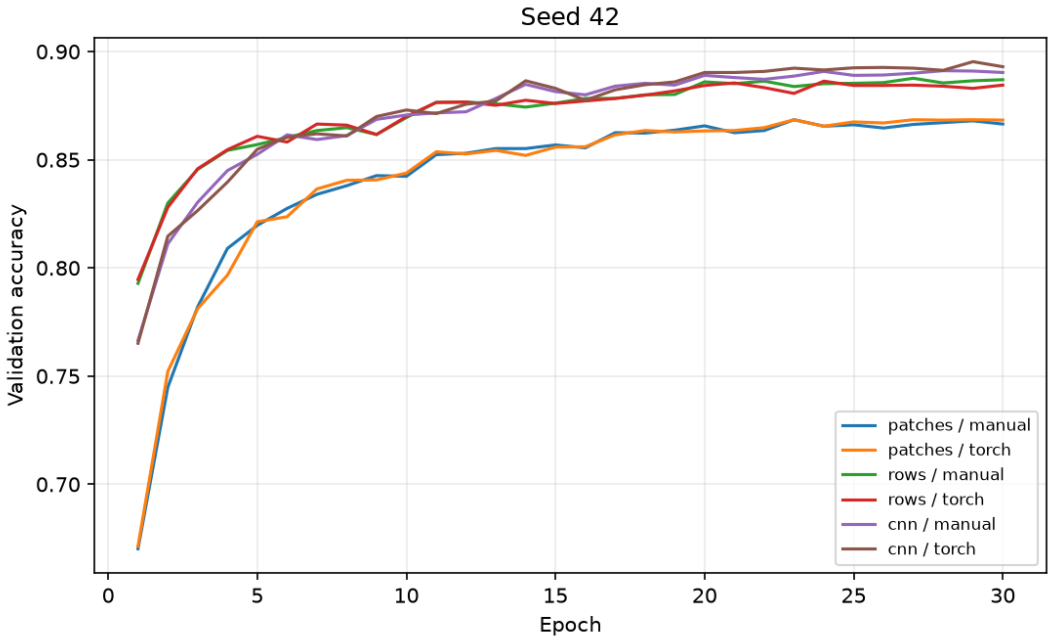
Khi dùng cùng bản attention tự viết, CNN-stem tăng độ chính xác so với token theo hàng +0.74 ± 0.42 điểm phần trăm.

Khi dùng cùng bản attention tự viết, CNN-stem tăng độ chính xác so với token theo ô ảnh +2.73 $\pm$ 0.31 điểm phần trăm.

Cấu hình E2 tốt nhất so với CNN E1 có chênh lệch -4.03 $\pm$ 0.46 điểm phần trăm. CNN-stem đưa đặc trưng cục bộ vào attention trước khi tổng hợp trên chuỗi. Phép so sánh thay đổi đồng thời số token, chiều đầu vào và số tham số, nên chỉ phản ánh hiệu quả của từng cách biểu diễn hoàn chỉnh.



Hình 3.1: Validation CE của sáu cấu hình E2 ở seed 42.



Hình 3.2: Validation accuracy khi thay attention backend và tokenizer.

Chương 4

E3: LSTM/GRU trên chuỗi ảnh

4.1 Biểu diễn ảnh và kiến trúc

Ảnh được biến thành chuỗi theo hai cách đã dùng trong E2, nhưng không qua embedding học được: hàng cho $T = 28, D = 28$; patch 4×4 cho $T = 49, D = 16$. Cả LSTM và GRU đều có một tầng, một chiều, hidden size $H = 128$, trạng thái đầu bằng zero. Baseline lấy hidden state cuối, dropout 0,1 rồi Linear $128 \to 10$.

Quét hàng giữ các quan hệ ngang trong một token và để mạng hồi tiếp xử lý quan hệ giữa các hàng. Quét patch tạo nhiều bước hơn, mỗi bước mô tả một vùng nhỏ hơn. Vì vậy, so sánh này thay đổi đồng thời độ dài chuỗi và kích thước đầu vào tại mỗi bước.

4.2 Các phương trình được sử dụng

Với LSTM, đặt $a_t = W_x x_t + W_h h_{t-1} + b$ rồi tách thành bốn phần [6]:

$$i_t = \sigma(a_t^i), \qquad f_t = \sigma(a_t^f), \qquad g_t = \tanh(a_t^g), \qquad o_t = \sigma(a_t^o), \tag{4.1}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t). \tag{4.2}$$

c_t có đường cập nhật cộng, còn h_t là trạng thái dùng cho output và hồi tiếp. Cổng quên lớn có thể giữ thông tin, nhưng kiến trúc này không tự bảo đảm mọi gradient đều được bảo toàn.

GRU dùng reset gate r_t, update gate z_t và trạng thái ứng viên n_t. Diagnostic khớp đúng cách PyTorch đặt reset gate sau phép chiếu hidden [7]:

$$r_t = \sigma(W_{ir} x_t + b_{ir} + W_{hr} h_{t-1} + b_{hr}), \tag{4.3}$$

$$z_t = \sigma(W_{iz} x_t + b_{iz} + W_{hz} h_{t-1} + b_{hz}), \tag{4.4}$$

$$n_t = \tanh(W_{in} x_t + b_{in} + r_t \odot (W_{hn} h_{t-1} + b_{hn})), \tag{4.5}$$

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}. \tag{4.6}$$

Trong công thức của PyTorch, reset gate tác động sau phép chiếu hidden. Đây là điểm khác với cách viết $W_{hn}(r_t \odot h_{t-1})$. GRU sử dụng một trạng thái hidden, còn LSTM tách hidden state và cell state.

PyTorch lưu hai vector bias cho mỗi tầng. Vì vậy, số tham số phần recurrent là $4H(D + H + 2)$ với LSTM và $3H(D + H + 2)$ với GRU; cộng thêm $10H + 10$ cho head. Giữ cùng H giúp so sánh rõ nhưng không làm hai mô hình có cùng capacity.

4.3 Kết quả và quan hệ không gian

Mô hình	Tham số	Val (%)	Test (%)	Giây/epoch
MLP	235,146	91.02	89.88 ± 0.14	1.67
CNN	421,738	93.03	92.69 ± 0.15	1.97
LSTM / rows	82,186	90.32	89.58 ± 0.05	1.77
GRU / rows	61,962	91.05	90.07 ± 0.17	1.73
LSTM / patches	76,042	88.83	87.65 ± 0.38	2.03
GRU / patches	57,354	88.97	88.00 ± 0.23	1.96

Bảng 4.1: E3 và hai baseline từ E1, trên cùng split.

Val/Test: accuracy (%). Tham số đếm từ kiến trúc; ±: SD qua ba seed. Các lượt chạy được liệt kê ở Chương 6; cách đo thời gian ở Chương 1.

CNN khai thác lân cận hai chiều và chia sẻ kernel; LSTM/GRU tích lũy thông tin theo đường quét. Khi chuyển từ cuối một hàng patch sang đầu hàng kế tiếp, hai phần tử kề nhau trong chuỗi chưa chắc là hàng xóm gần nhất trên ảnh. Đường quét vì vậy giữ thứ tự một chiều và chỉ bảo toàn một phần quan hệ lân cận hai chiều.

Ảnh hưởng của cách quét được khảo sát bằng cách đánh giá checkpoint trên cùng ảnh validation với thứ tự gốc, đảo ngược và một hoán vị cố định. Mô hình giữ nguyên trọng số trong ba phép đo. Gradient theo vị trí chuỗi được đo đồng thời để mô tả đường truyền loss. LSTM / rows trừ CNN E1: -3.11 ± 0.18 điểm phần trăm.

GRU / rows trừ CNN E1: -2.62 ± 0.13 điểm phần trăm.

LSTM / patches trừ CNN E1: -5.04 ± 0.35 điểm phần trăm.

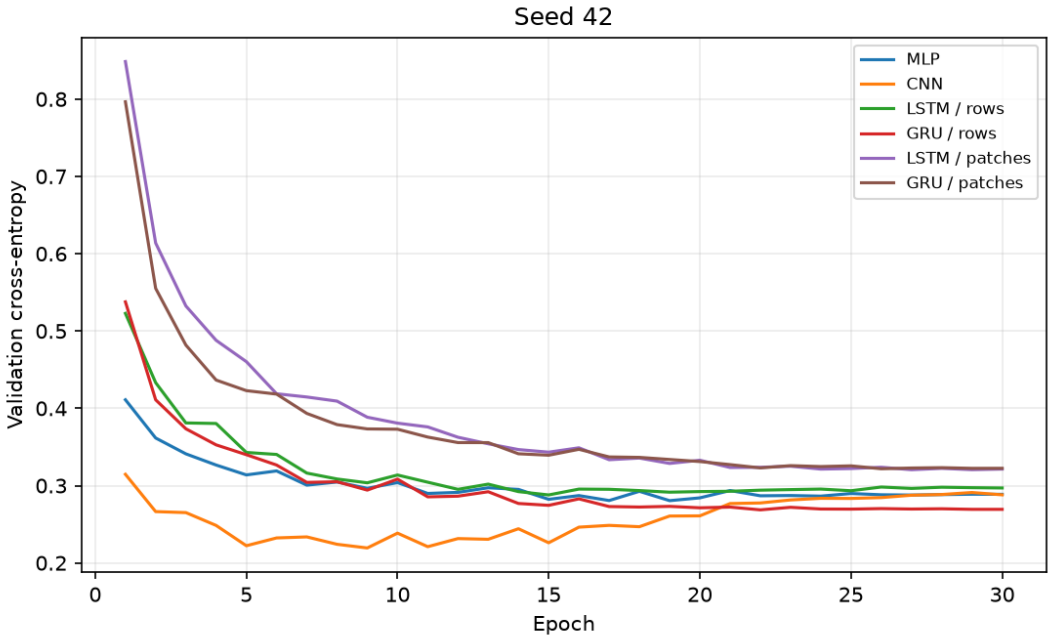
GRU / patches trừ CNN E1: -4.69 ± 0.32 điểm phần trăm.

Cách quét theo hàng và theo ô thay đổi đồng thời số bước và số pixel mỗi token. Kết quả phản ánh ảnh hưởng kết hợp của hai yếu tố.

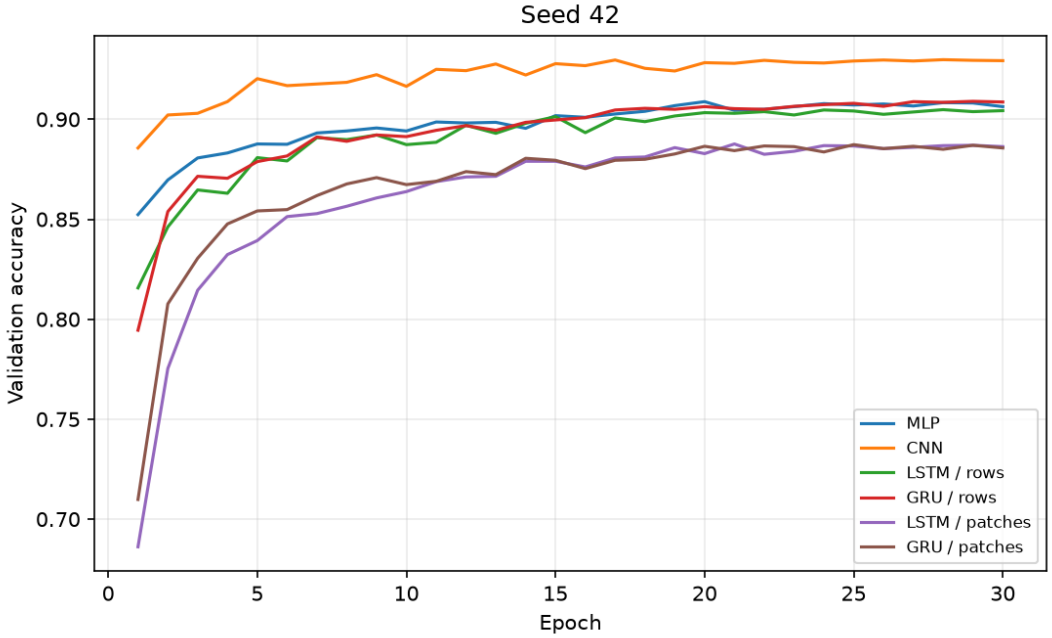
LSTM rows trừ patches: +1.93 ± 0.42 điểm phần trăm.

GRU rows trừ patches: +2.07 ± 0.22 điểm phần trăm.

GRU rows trừ LSTM rows: +0.49 ± 0.21 điểm phần trăm. Quét theo hàng giữ trọn thông tin theo chiều ngang trong một token và cho chuỗi ngắn hơn quét theo ô. Đây là hai đặc điểm có thể giúp mạng tổng hợp thông tin trên tập ảnh này.



Hình 4.1: Validation CE của các mạng hồi tiếp và baseline E1 ở cùng seed.



Hình 4.2: So sánh accuracy của E3 với MLP/CNN.

Chương 5

Mở rộng E3: Điều kiện truyền gradient

5.1 Vấn đề và giả thuyết

Phần này khảo sát ảnh hưởng của khởi tạo, regularization và cách tổng hợp trạng thái lên LSTM. Chuỗi hàng và patch có độ dài lần lượt 28 và 49 bước, phù hợp để kiểm tra các thay đổi này trên bài toán ảnh nhỏ. Phân tích bắt đầu từ tích Jacobian của mạng hồi tiếp, sau đó đối chiếu với gradient và kết quả thực nghiệm.

Với trạng thái hồi quy tổng quát $s_t = F_\theta(s_{t-1}, x_t)$ và $J_t = \partial s_t / \partial s_{t-1}$, đóng góp từ loss cuối chuỗi thỏa

$$\frac{\partial \ell_T}{\partial s_t} = J_{t+1}^\top J_{t+2}^\top \cdots J_T^\top \frac{\partial \ell_T}{\partial s_T}, \qquad \left\| \frac{\partial \ell_T}{\partial s_t} \right\| \le \prod_{k=t+1}^{T} \|J_k\|_2 \left\| \frac{\partial \ell_T}{\partial s_T} \right\|. \tag{5.1}$$

Tích nhiều phép co có thể làm gradient nhỏ; tích nhiều phép khuếch đại có thể làm nó lớn. Cận theo norm là một cách phân tích, không mô tả chính xác mọi hướng gradient. Với LSTM, s_t cần gồm cả h_t, c_t, không chỉ một biến hidden [4].

5.2 Khởi tạo và regularization trực giao

Với từng gate, ma trận hồi quy vuông $W_g \in \mathbb{R}^{H \times H}$ được khởi tạo trực giao ở các cấu hình tương ứng. Nếu $W_g^\top W_g = I$ thì $\|W_g v\|_2 = \|v\|_2$: riêng phép biến đổi tuyến tính đó không làm thay đổi norm. Điều này không loại bỏ tác động của sigmoid, tanh, các gate và tích Jacobian toàn cell.

Tập các ma trận trực giao là trường hợp vuông của Stiefel manifold. Thay vì dùng optimizer có ràng buộc trên đa tạp, code dùng một penalty mềm trong không gian tham số thông thường:

$$\mathcal{R}_{orth} = \frac{1}{GH} \sum_{g=1}^{G} \left\| W_g^\top W_g - I \right\|_F^2, \qquad \mathcal{L} = \mathcal{L}_{CE} + \lambda \mathcal{R}_{orth}, \tag{5.2}$$

trong đó $G = 4$ cho LSTM, $H = 128$ và $\lambda = 10^{-3}$ ở các cấu hình có regularization. Penalty được tính riêng cho bốn ma trận gate $H \times H$, rồi lấy trung bình.

Đặt $A = W^\top W - I = A^\top$. Vi phân cho một gate là

$$d \, \|A\|_F^2 = 2 \operatorname{tr}(A \, dA) = 2 \operatorname{tr}\big[A(dW^\top W + W^\top dW)\big] \tag{5.3}$$

$$= 4 \operatorname{tr}\big[(WA)^\top dW\big], \tag{5.4}$$

do đó

$$\nabla_{W_g}\mathcal{R}_{orth} = \frac{4}{GH}W_g(W_g^\top W_g - I). \tag{5.5}$$

Công thức được kiểm tra với autograd. Penalty tạo chi phí tính toán thêm và có thể làm giảm mức linh hoạt của nghiệm. AdamW vẫn có weight decay cố định trên mọi tham số, nên nó đồng thời kéo trọng số về zero; không được mô tả quá trình train như duy trì trực giao chính xác.

Đây là penalty mềm trong không gian Euclid, không dùng diffusion maps, đồ thị dữ liệu hoặc optimizer manifold.

5.3 Bias cổng nhớ theo độ dài chuỗi

Ở cấu hình chrono, đặt $b_f = \log(T-1)$, $b_i = -\log(T-1)$; bias các gate khác bằng zero. Chỉ thay `bias_ih`; giữ `bias_hh=0` để tổng bias đúng giá trị dự định. Biến thể này dùng *một timescale cố định*, thay cho cách lấy timescale ngẫu nhiên trong công trình chrono initialization gốc [5].

Khi bỏ qua đóng góp từ input/hidden tại lúc khởi tạo,

$$f_t \approx \sigma(\log(T-1)) = 1 - \frac{1}{T}, \qquad i_t \approx \frac{1}{T}. \tag{5.6}$$

Giữ các gate cố định để chỉ xét đường trực tiếp $c_{t-1} \to c_t$, ta có

$$\left.\frac{\partial c_T}{\partial c_t}\right|_{\text{đường trực tiếp}} = \prod_{k=t+1}^{T} \operatorname{diag}(f_k). \tag{5.7}$$

Nếu mỗi $f_k \approx 1 - 1/T$ thì độ suy giảm trên khoảng T bước gần $(1-1/T)^T \approx e^{-1}$, thay vì 2^{-T} khi $f_k \approx 0,5$. Đây chỉ là xấp xỉ để giải thích khởi tạo. Đạo hàm tổng còn có các đường qua hidden và đạo hàm của gate; không thể đồng nhất nó với tích trên.

Cổng quên gần 1 đi kèm cổng nhập nhỏ có thể khiến mô hình cập nhật thông tin mới chậm; bias lớn cũng có thể đưa sigmoid vào vùng bão hòa. Các bias tiếp tục được cập nhật trong quá trình huấn luyện; chrono được đánh giá như một cấu hình riêng.

5.4 Mean pooling và clipping

Baseline dự đoán từ h_T. Biến thể mean pooling dùng $\bar{h} = T^{-1}\sum_t h_t$ trước head. Bỏ qua dropout trong biểu thức minh họa, nếu $z = W_o\bar{h} + b$ và y là vector one-hot thì đường gradient trực tiếp tới mỗi trạng thái là

$$\left.\frac{\partial \ell}{\partial h_t}\right|_{\text{head}} = \frac{1}{T}W_o^\top(p - y). \tag{5.8}$$

Đạo hàm tổng còn gồm các đường hồi quy về sau. Lợi ích tiềm năng là một trạng thái sớm không chỉ phụ thuộc vào việc tín hiệu đi qua toàn bộ chuỗi để tới head. Đổi lại, trung bình có thể làm mờ vị trí và trộn vùng nền vào biểu diễn. Learned position không được thêm cho E3; mạng hồi

tiếp tự phụ thuộc thứ tự quét.

Clipping global norm được dùng cho tất cả baseline lẫn cải tiến:

$$g \leftarrow g \min\left(1, \frac{\tau}{\|g\|_2}\right), \qquad \tau = 1. \tag{5.9}$$

Clipping giới hạn norm của gradient trước bước cập nhật, giúp kiểm soát exploding gradient. Cơ chế này không khuếch đại các gradient đang suy giảm [4].

5.5 Ablation và cách kiểm tra

Các cấu hình dưới đây dùng LSTM theo hàng, giữ $H = 128$, cách chia dữ liệu và ngân sách huấn luyện. "Orth init" thay khởi tạo. "Orth reg" thêm penalty vào bản orth init. "Combined" dùng trực giao ban đầu, penalty, chrono và mean pooling. Hai bản "no reg" và "no mean" bỏ riêng một thành phần từ combined; các thành phần khác giữ nguyên.

Mô hình	Tham số	Val (%)	Test (%)	Giây/epoch
LSTM / rows	82,186	90.32	89.58 ± 0.05	1.77
LSTM + orthogonal init	82,186	90.20	89.35 ± 0.17	1.77
LSTM + orthogonal init/reg	82,186	90.07	89.47 ± 0.24	3.05
LSTM + chrono bias	82,186	90.23	89.28 ± 0.19	1.78
LSTM + mean pooling	82,186	89.28	88.32 ± 0.09	1.76
LSTM combined	82,186	89.17	87.92 ± 0.29	2.97
Combined without regularizer	82,186	89.07	87.91 ± 0.32	1.79
Combined without mean pooling	82,186	90.17	89.30 ± 0.23	3.03

Bảng 5.1: Baseline và bảy cấu hình mở rộng. Tất cả có cùng số tham số trainable.

Val/Test: accuracy (%). Tham số đếm từ kiến trúc; ±: SD qua ba seed. Các lượt chạy được liệt kê ở Chương 6; cách đo thời gian ở Chương 1.

Chênh lệch orth init–baseline khảo sát khởi tạo; orth reg–orth init khảo sát penalty. Chrono và mean pooling được so riêng với baseline. Hai ablation no reg và no mean đánh giá đóng góp của từng thành phần trong tổ hợp. Chênh lệch trung bình và độ lệch chuẩn được tính từ các kết quả ghép theo cùng seed.

Phép đo unroll dùng trọng số của checkpoint native và công thức cell tương ứng, giữ gradient tại từng trạng thái trung gian. Loss dùng tổng theo batch để gradient từng mẫu không bị chia bởi batch size; sau đó lấy trung bình norm qua mẫu. Cách tính này cho phép khảo sát trực tiếp các trạng thái trong đồ thị BPTT.

Trên cùng 256 ảnh validation theo quy tắc định trước, diagnostic ghi $\|\partial \ell / \partial h_t\|$, $\|\partial \ell / \partial c_t\|$, gradient theo input token, giá trị gate trung bình và singular values của từng W_g. Các norm mô tả đường truyền loss của checkpoint đã học. Gradient nhỏ ở pixel nền cần được đọc cùng accuracy, CE và vị trí không gian của ảnh. Khởi tạo trực giao so với baseline làm thay đổi accuracy -0.23 ± 0.13 điểm phần trăm. Thêm penalty vào bản khởi tạo trực giao thay đổi +0.12 ± 0.13 điểm

phần trăm, trong khi thời gian/epoch tăng từ 1.77s lên 3.05s.

LSTM + chrono bias so với baseline: -0.29 ± 0.19 điểm phần trăm.

LSTM + mean pooling so với baseline: -1.26 ± 0.11 điểm phần trăm.

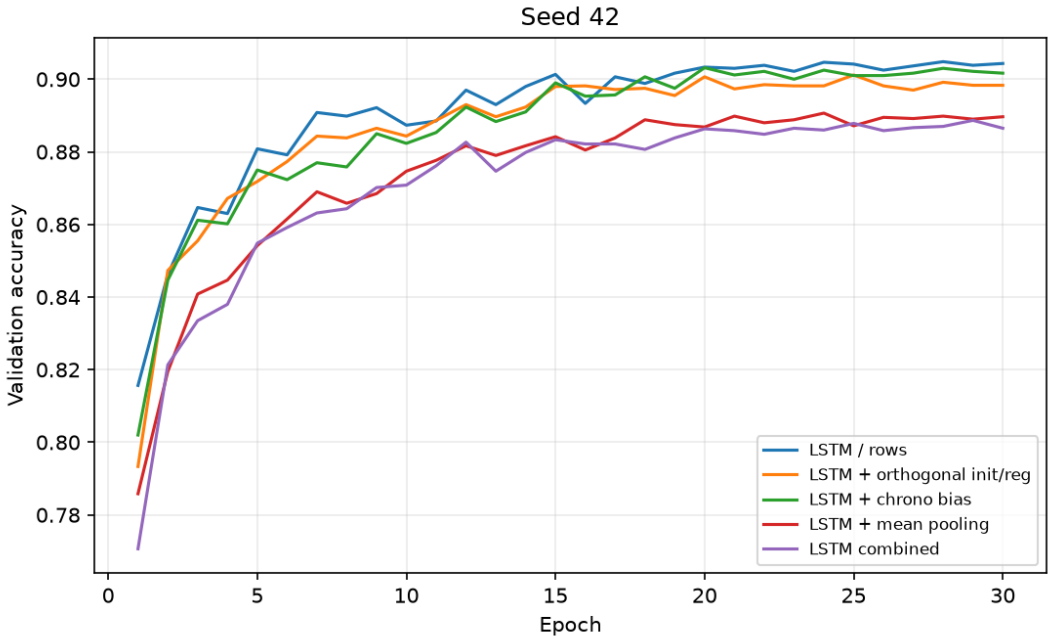
Tổ hợp trừ baseline: -1.66 ± 0.30 điểm phần trăm. Bỏ mean pooling khỏi tổ hợp thay đổi accuracy +1.39 ± 0.52 điểm phần trăm; bỏ regularizer thay đổi -0.01 ± 0.35 điểm phần trăm và giảm thời gian/epoch 39.8% theo các median đã đo. Mean pooling có ảnh hưởng rõ hơn trong hai ablation này; phần tính penalty chiếm thêm thời gian dù chênh lệch accuracy khi bỏ penalty khá nhỏ.

Cả bảy biến thể mở rộng có test accuracy trung bình thấp hơn baseline. Trong thiết lập này, các thay đổi chưa mang lại cải thiện accuracy. Mean pooling gộp đồng đều cả những trạng thái đầu chuỗi có ít ngữ cảnh; việc trộn các trạng thái này có thể làm loãng biểu diễn trước head.

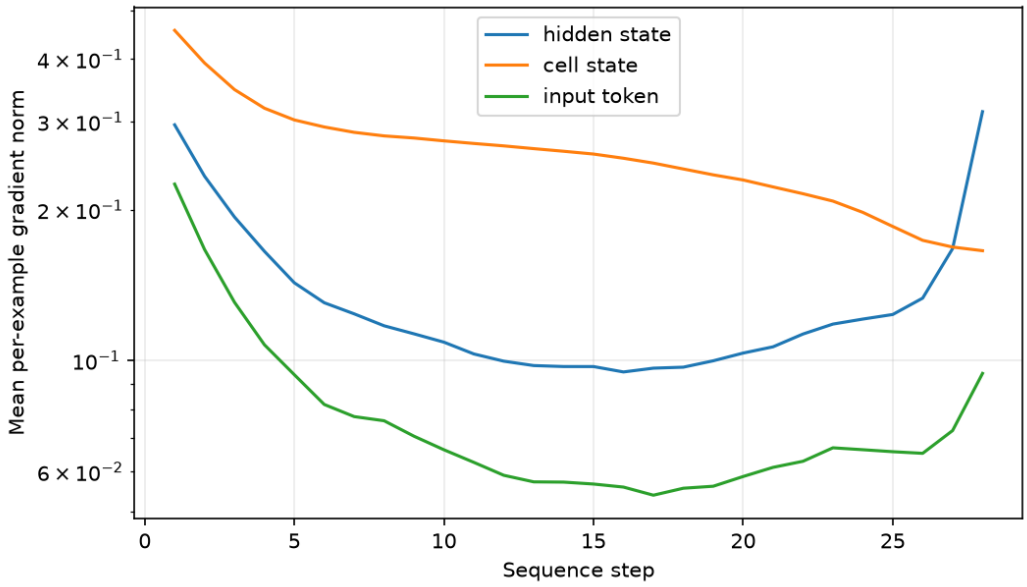
LSTM / rows trên 256 ảnh validation: original accuracy 89.71%; reversed accuracy 20.96%; permuted accuracy 44.53%. Hidden gradient norm trung bình tại bước 1/bước 28 là 0.293/0.304.

LSTM combined trên 256 ảnh validation: original accuracy 88.80%; reversed accuracy 30.73%; permuted accuracy 42.97%. Hidden gradient norm trung bình tại bước 1/bước 28 là 0.34/0.0192.

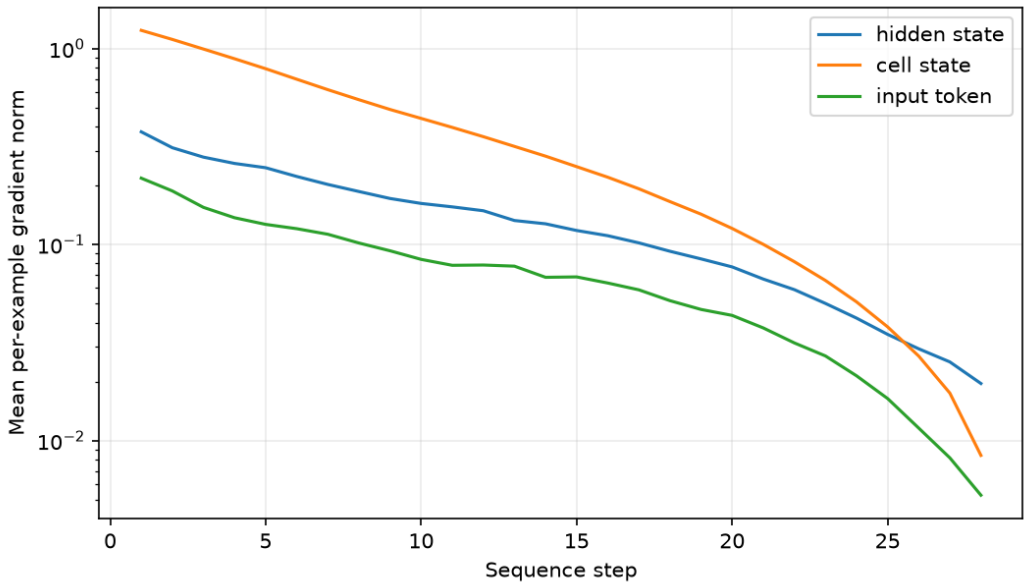
Độ chính xác giảm khi đảo hoặc tráo thứ tự, cho thấy mô hình nhạy với cách quét ảnh. Các chuỗi biến đổi nằm ngoài phân phối huấn luyện, nên phép đo này chưa xác lập khả năng học phụ thuộc dài hạn. Phép gộp trung bình nối bộ phân loại trực tiếp với từng trạng thái ẩn, còn phép lấy trạng thái cuối chỉ tạo một đường nối trực tiếp. Gradient đo tại mỗi bước vẫn bao gồm đóng góp qua các trạng thái hồi tiếp về sau.



Hình 5.1: Validation accuracy của các thay đổi chính ở cùng seed.



Hình 5.2: Gradient theo vị trí của LSTM baseline trên validation.



Hình 5.3: Gradient của tổ hợp cải tiến, cùng cách chọn validation subset.

Chương 6

Tái lập và giới hạn

6.1 Mã nguồn và tệp kết quả

Repo tách mô hình thành `src/E1/`, `src/E2/`, `src/E3/`; phần dùng chung nằm trong `src/common/`. Mỗi run có `run.json`, `history.csv`, `best.pt`, `last.pt` và `metrics.json`. Đánh giá test sinh thêm prediction CSV, confusion matrix, ảnh sai và chỉ số theo lớp. Hyperparameter được khai báo trong một file JSON gồm 20 preset.

Checkpoint cuối epoch lưu model, AdamW, cosine scheduler, lịch sử và trạng thái RNG của Python, NumPy, PyTorch, CUDA, shuffle generator. Resume từ epoch hoàn tất gần nhất, không từ batch đang dở. Fingerprint kiểm tra code, config, seed, dữ liệu và normalization; môi trường torch/CUDA cũng được đối chiếu. Khóa thư mục ngăn hai tiến trình ghi cùng run. Cơ chế này không hứa khả năng tái lập bitwise giữa các phần cứng khác nhau.

`scripts/export_report.py` tổng hợp kết quả thành bảng LaTeX, CSV/JSON và hình vẽ. Chương trình đối chiếu cấu hình, seed, split và checkpoint trước khi gộp các lượt chạy.

6.2 Cấu hình và các lượt chạy

Thực nghiệm hoàn tất 60 lượt huấn luyện và 60 lượt đánh giá test. Bảng dưới liệt kê các seed của từng cấu hình.

Cấu hình	Seed train	Seed test
Softmax	42,43,44	42,43,44
MLP	42,43,44	42,43,44
CNN	42,43,44	42,43,44
patches / manual	42,43,44	42,43,44
patches / torch	42,43,44	42,43,44
rows / manual	42,43,44	42,43,44
rows / torch	42,43,44	42,43,44
cnn / manual	42,43,44	42,43,44
cnn / torch	42,43,44	42,43,44
LSTM / rows	42,43,44	42,43,44
GRU / rows	42,43,44	42,43,44
LSTM / patches	42,43,44	42,43,44
GRU / patches	42,43,44	42,43,44
LSTM + orthogonal init	42,43,44	42,43,44

Cấu hình	Seed train	Seed test
LSTM + orthogonal init/reg	42,43,44	42,43,44
LSTM + chrono bias	42,43,44	42,43,44
LSTM + mean pooling	42,43,44	42,43,44
LSTM combined	42,43,44	42,43,44
Combined without regularizer	42,43,44	42,43,44
Combined without mean pooling	42,43,44	42,43,44

Các kiểm thử bao gồm forward/gradient của MSA, thứ tự token, unroll LSTM/GRU, đạo hàm regularizer, bias chrono, split và khôi phục trạng thái khi resume. Môi trường chạy, log và checksum được ghi trong `docs/TRAINING_EXECUTION.md` cùng thư mục kết quả.

Ba seed cho phép quan sát biến thiên giữa các lượt khởi tạo. Phạm vi so sánh giới hạn ở một tập dữ liệu, ngân sách 30 epoch và các siêu tham số đã chọn. Các chuỗi 28/49 bước cũng ngắn hơn nhiều so với những bài toán phụ thuộc dài hạn.

6.3 Kết luận

Trong 13 cấu hình cơ bản, CNN có test accuracy trung bình cao nhất, đạt 92.69%. CNN-stem cho kết quả tốt nhất trong nhóm E2, còn biểu diễn theo hàng phù hợp hơn patch đối với cả LSTM và GRU. Bảy biến thể mở rộng E3 đều có accuracy trung bình dưới baseline LSTM rows. Các ablation cho thấy mean pooling ảnh hưởng lớn đến accuracy của tổ hợp, trong khi penalty trực giao làm tăng chi phí huấn luyện.

Phạm vi hỗ trợ của AI được trình bày trong `AI_USE_REPORT.md`.

Tài liệu tham khảo

[1] Lê Thành Sách, Nguyễn Quốc Minh. *Bài tập E1–E3: Học sâu và ứng dụng trong thị giác máy tính, CO5085*. Đề học kỳ 261 (2026–2027).

[2] H. Xiao, K. Rasul, R. Vollgraf. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. arXiv:1708.07747, 2017. https://arxiv.org/abs/1708.07747. Dữ liệu: https://github.com/zalandoresearch/fashion-mnist.

[3] A. Vaswani và cộng sự. *Attention Is All You Need*. NeurIPS, 2017. https://arxiv.org/abs/1706.03762.

[4] R. Pascanu, T. Mikolov, Y. Bengio. *On the difficulty of training recurrent neural networks*. ICML, 2013; arXiv:1211.5063. https://arxiv.org/abs/1211.5063.

[5] C. Tallec, Y. Ollivier. *Can recurrent neural networks warp time?* ICLR, 2018. https://arxiv.org/abs/1804.11188.

[6] PyTorch. *LSTM: API and equations*. https://docs.pytorch.org/docs/2.9/generated/torch.nn.LSTM.html.

[7] PyTorch. *GRU: API and implementation notes*. https://docs.pytorch.org/docs/2.9/generated/torch.nn.GRU.html.

[8] PyTorch. *Previous versions: torch 2.10.0 / torchvision 0.25.0*. https://pytorch.org/get-started/previous-versions/.